

SIMATSENGINEERING(SAVEETHASCHOOLOFENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5-Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 - Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse a real-world problem.		
Student Name:	DHIVYADHARSAN M.S	Reg. No.:	192511465
Section/Batch:	2025	Date:	04/09/2026

Code of Conduct

I, DHIVYADHARSAN M.S (Name/RegNo)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: DHIVYADHARSAN M.S

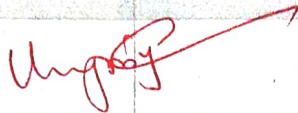
Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answers using Flowchart-to-code conversions should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section/Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature:	Signature:
Date:	Date:	Date:

Name: M.S. Dhivyaadharsan

Reg No: 192511465

Course Name: Data Structure

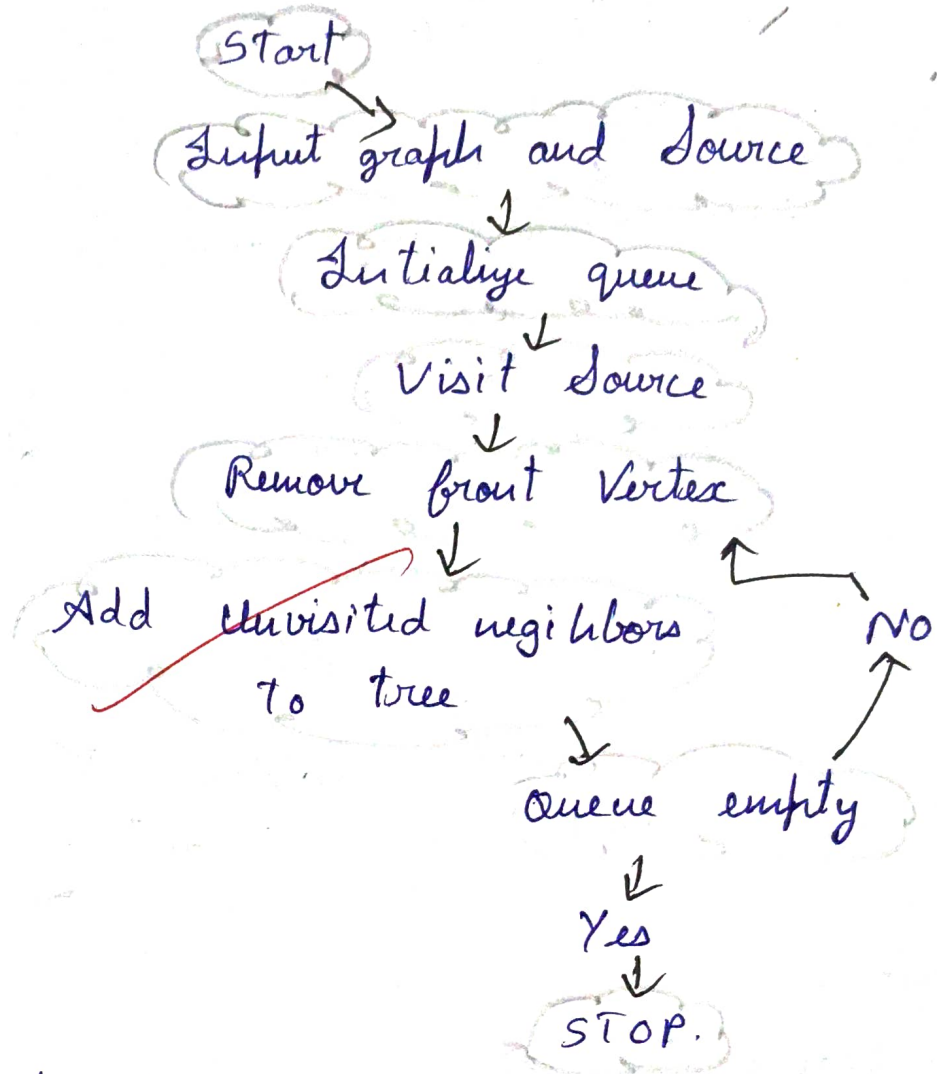
Course Code: CSA0305

Date: 21/8/26

Flowchart to Code

CO5 - AT2

- 1) Convert The flowchart for generating a BFS tree into code.



Ans. Code

```
#include <stdio.h>
int main() {
    int a[10][10], q[10], v[10] = {0}, u, s, f = 0, x = 0, i, j;
    scanf("%d", &u);
    for (i = 0; i < u; i++) for (j = 0; j < u; j++)
        scanf("%d", &a[i][j]);
    scanf("%d", &s); q[x++] = s; v[s] = 1;
```

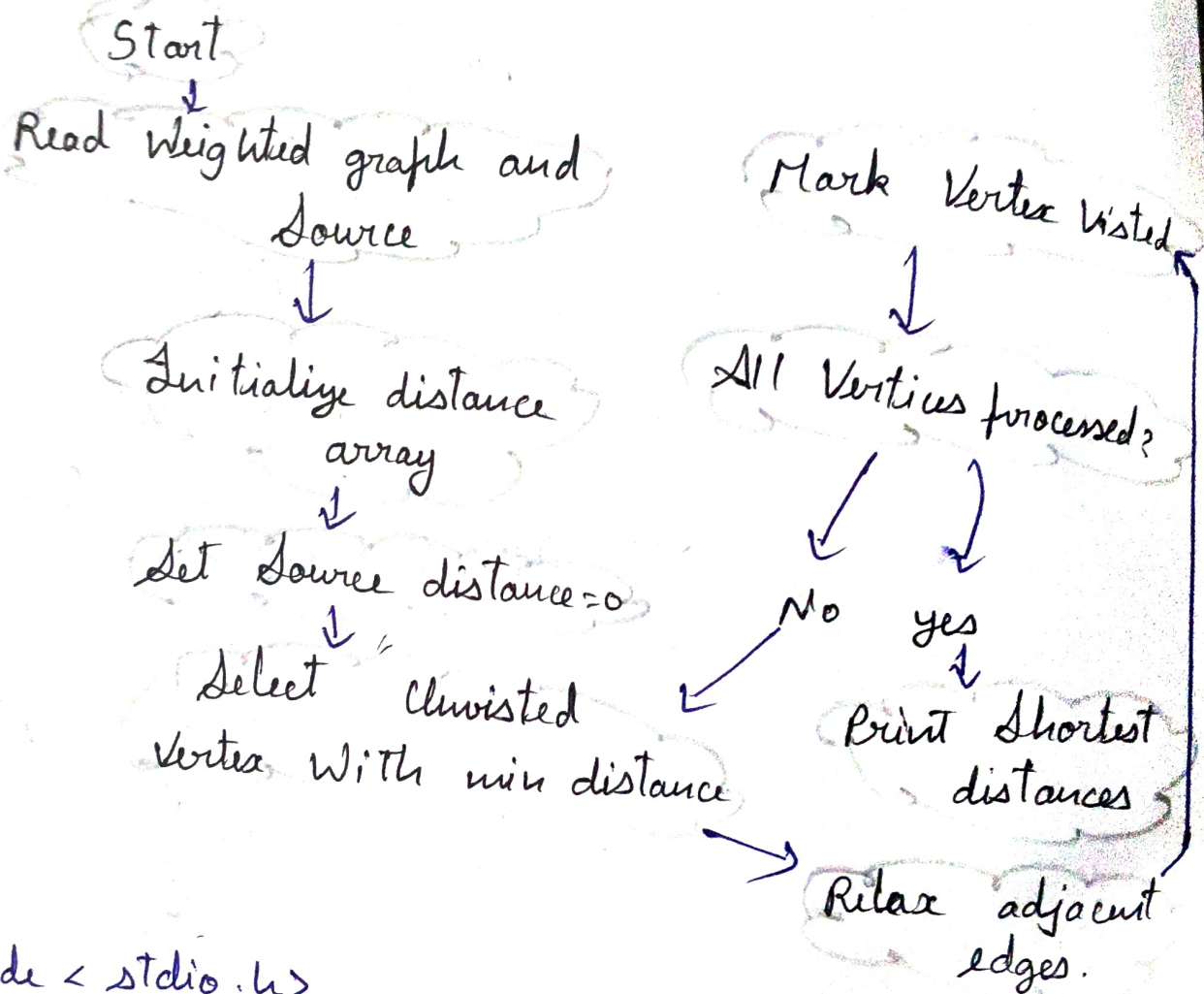


```

while (f < n) { s = q[f++]; printf ("%d", s);
for (i = 0; i < n; i++) if (a[s][i] &amp; d != v[i] v[i] = a[s][i] + d; }
}

```

2) Convert the flowchart for Dijkstra's shortest path algorithm into code.



Ans Code

```

#include <stdio.h>
int main () {
int a[10][10], d[10], v[10] = {0}, n, s, i, j, u, m;
scanf ("%d", &n);
for (i = 0; i < n; i++) for (j = 0; j < n; j++)
scanf ("%d", &a[i][j]);
scanf ("%d", &s);
for (i = 0; i < n; i++) d[i] = 999;

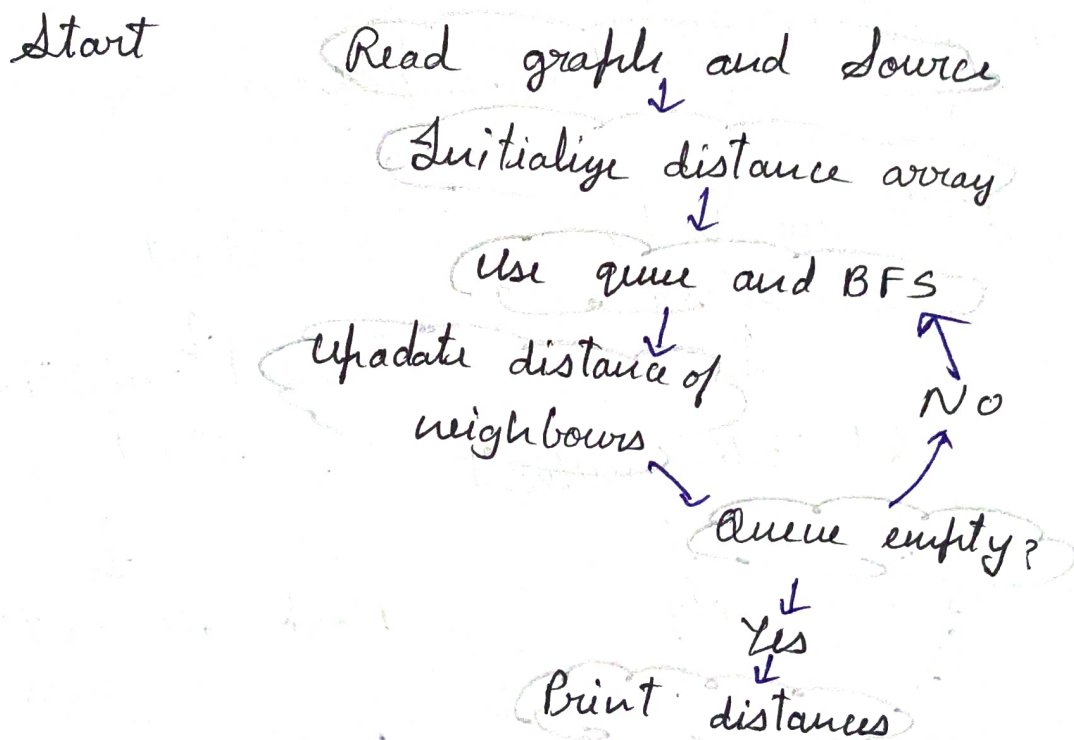
```

```

d[u] = 0;
for (i = 0; i < n; i++) { m = 999; }
for (j = 0; j < n; j++) if (!v[j] && d[j] < m) m = d[j] + a[u][j];
v[u] = 1;
for (j = 0; j < n; j++) if (a[u][j] && d[j] > d[u] + a[u][j])
    d[j] = d[u] + a[u][j];
for (i = 0; i < n; i++) printf("%d ", d[i]);
}

```

3) Convert The flowchart for shortest path in an Unweighted graph into Code.



Ans Code

```

#include <stdio.h>
int main() {
    int a[10];
    [0], d[10], v[10], u[10] = {0}, n, s, f = 0, m = 0, u, i, j;
    scanf("%d", &n);
}

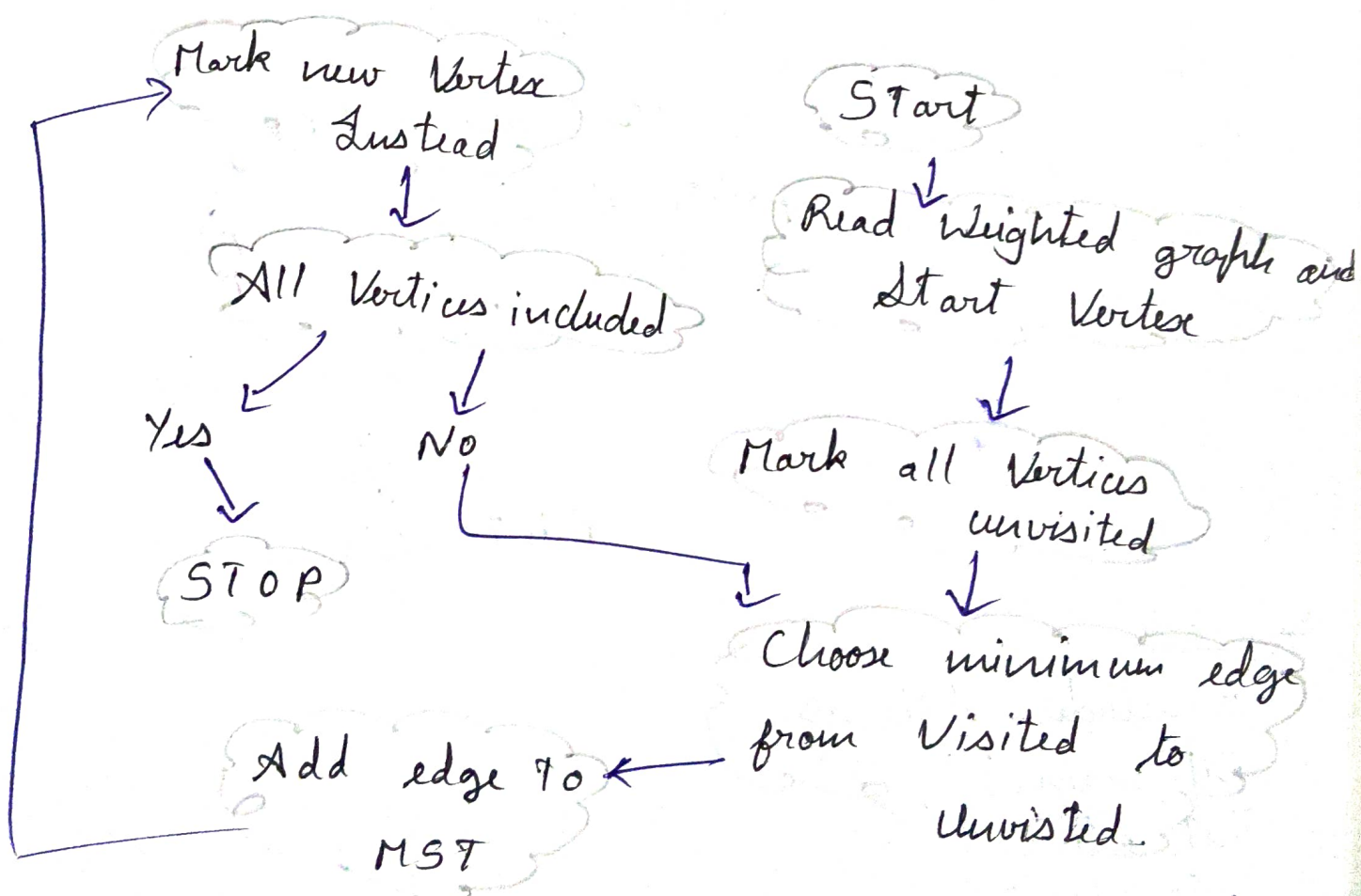
```

```

for (i=0; i<n; i++) for (j=0; j<n; j+
+ ) scanf ("%d", &a[i][j]);
scanf ("%d", &s);
for (i=0; i<n; i++) d[i]=1;
q[n++] = s; v[s]=1; d[s]=0;
while (f<n) {u=q[f++];
for (i=0; i<n; i++) if (a[u][i]&&
v[i]) v[i]=1; d[i]=d[u]+1; q[n++] = i;}
for (i=0; i<n; i++) printf ("%d", d[i]);
3

```

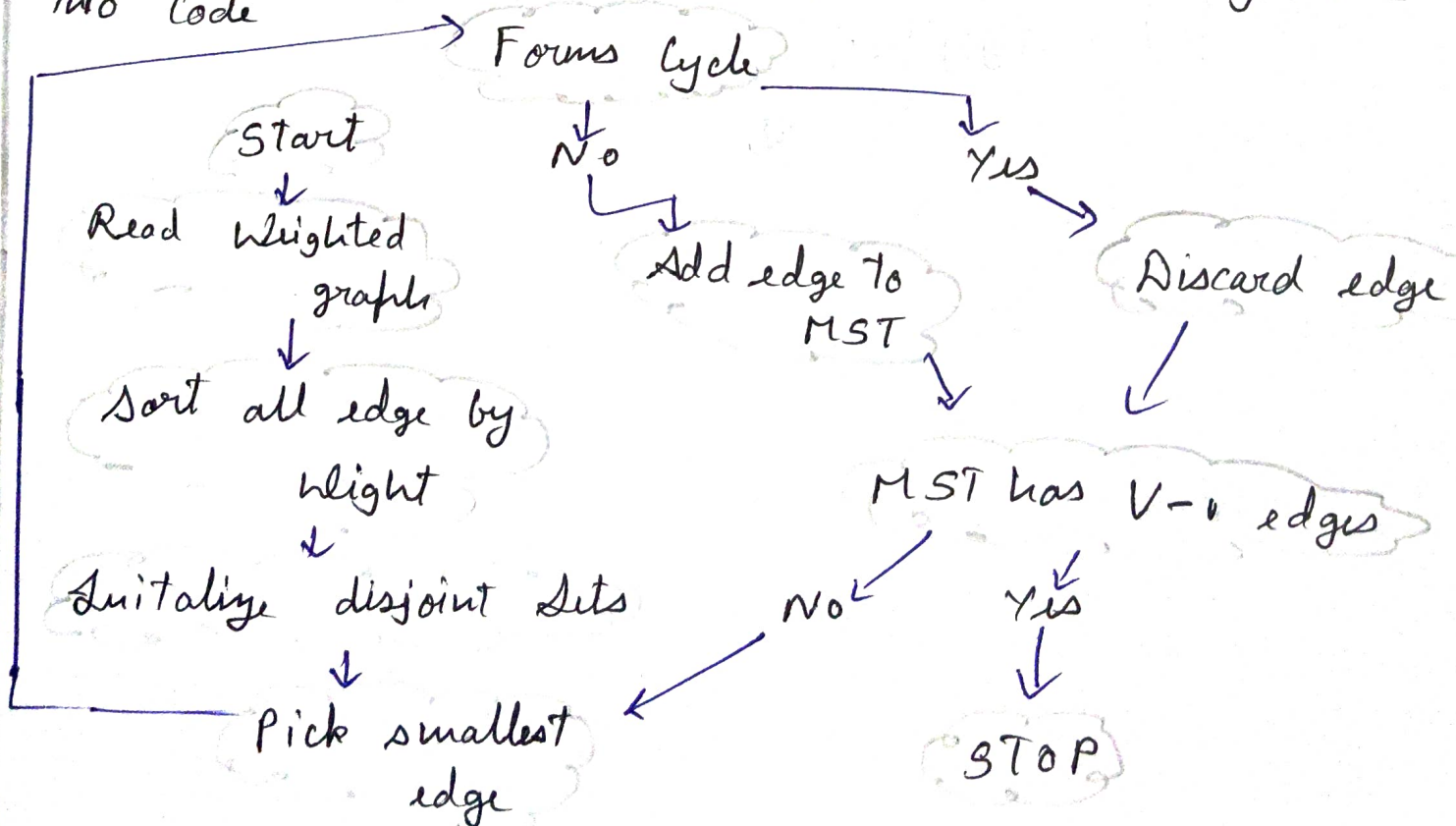
4) Convert The flowchart for prim's algorithm into Code.



Code

```
#include <stdio.h>
int main () {
    int a [10] [10], v [10] = {1, 3, 4, 1, 5, 7, 8, 9, 10, 11};
    scanf ("%d", &n);
    for (i=0; i<n; i++) for (j=0; j<n; j++)
        scanf ("%d", &a[i][j]);
    while (c < n-1) { m = 999;
        for (i=0; i<n; i++) if (v[i] for (j=0; j<n; j++)
            if (i < j & a[i][j] < m) m = a[i][j], x = j;
        v[x] = 1; printf ("%d", m); c++; }
}
```

5) Convert the flowchart for Kruskal's algorithm into code



Ans Code

```
#include <stdio.h>
int p[10];
int f(int x) { return p[x] == x ? x : p[x] = f(p[x]); }
int main () {
    int a[10][10], n, i, j, x, y, m, c = 0;
    scanf ("%d", &n);
    for (i = 0; i < n; i++) { p[i] = i; for (j = 0; j < n; j++)
        scanf ("%d", &a[i][j]); }
    while (c < n - 1) { m = 999;
        for (i = 0; i < n; i++) for (j = i + 1; j < n; j++)
            if (a[i][j] && a[j][i] < m) m = a[i][j], x = i, y = j;
        x = f(x); y = f(y);
        if (x != y) p[x] = y, printf ("%d", m), c++;
        a[x][y] = a[y][x] = 999; }
}
```

Ugma